Chapter ML:III

III. Linear Models

- Logistic Regression
- Overfitting
- Regularization

©STEIN 2005-2020

Logistic Regression

Binary Classification Problems

Setting:

- The feature space $X \subseteq \mathbf{R}^p$ is an inner product space.
- $C = \{0, 1\}$ is a set of two classes. Similarly: $\{-1, 1\}$, $\{\ominus, \oplus\}$, $\{\text{no}, \text{yes}\}$, etc.
- $c : X \to C$ is the (unknown) ideal classifier for X.
- $D = \{(\mathbf{x}_1, c(\mathbf{x}_1)), \ldots, (\mathbf{x}_n, c(\mathbf{x}_n))\} \subseteq X \times C$ is a set of examples.

Todo:

- Approximate c, which is implicitly given via D, with a logistic model function.

©STEIN 2005-2020

Logistic Regression

Binary Classification Problems

Setting:

- The feature space $X \subseteq \mathbf{R}^p$ is an inner product space.
- $C = \{0, 1\}$ is a set of two classes. Similarly: $\{-1, 1\}$, $\{\ominus, \oplus\}$, $\{\text{no}, \text{yes}\}$, etc.
- $c : X \to C$ is the (unknown) ideal classifier for X.
- $D = \{(\mathbf{x}_1, c(\mathbf{x}_1)), \ldots, (\mathbf{x}_n, c(\mathbf{x}_n))\} \subseteq X \times C$ is a set of examples.

Todo:

- Approximate c, which is implicitly given via D, with a logistic model function.

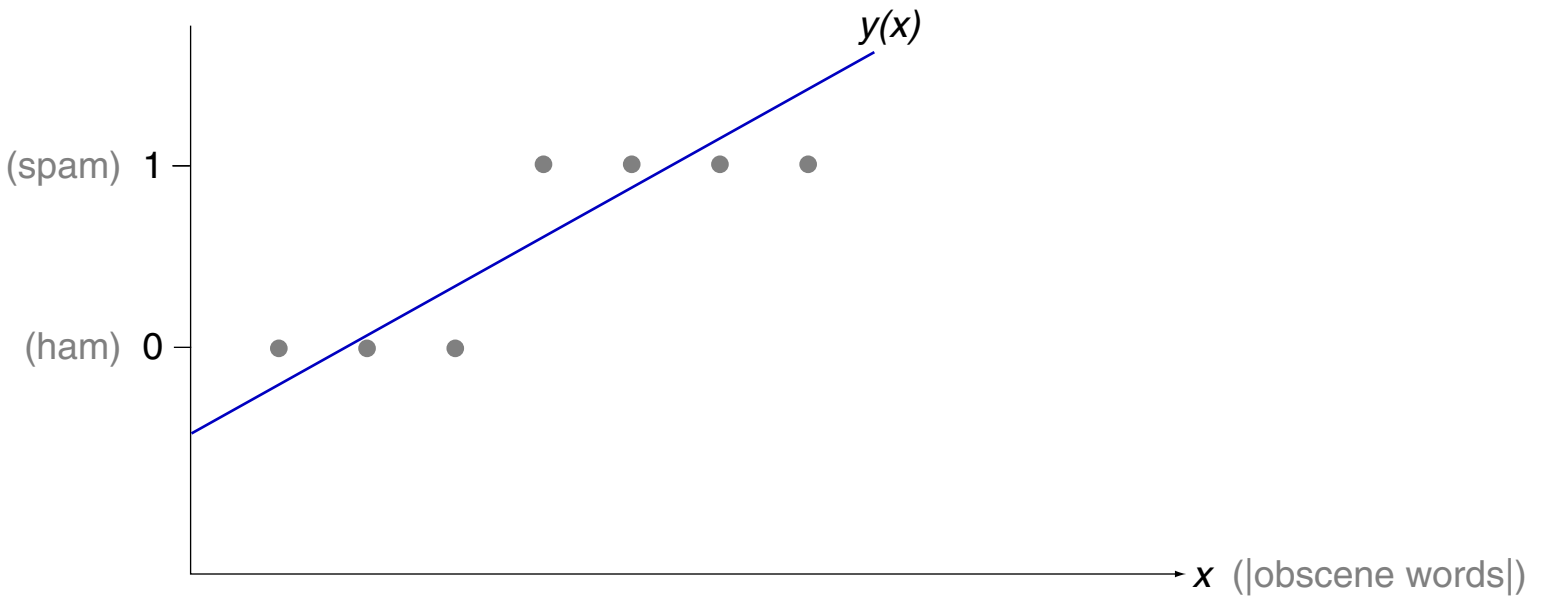
Examples for binary classification problems:

- Is an email spam or ham?
- Is a patient infected or healthy?
- Is a bank customer creditworthy or not?

©STEIN 2005-2020

Logistic Regression

Linear Regression

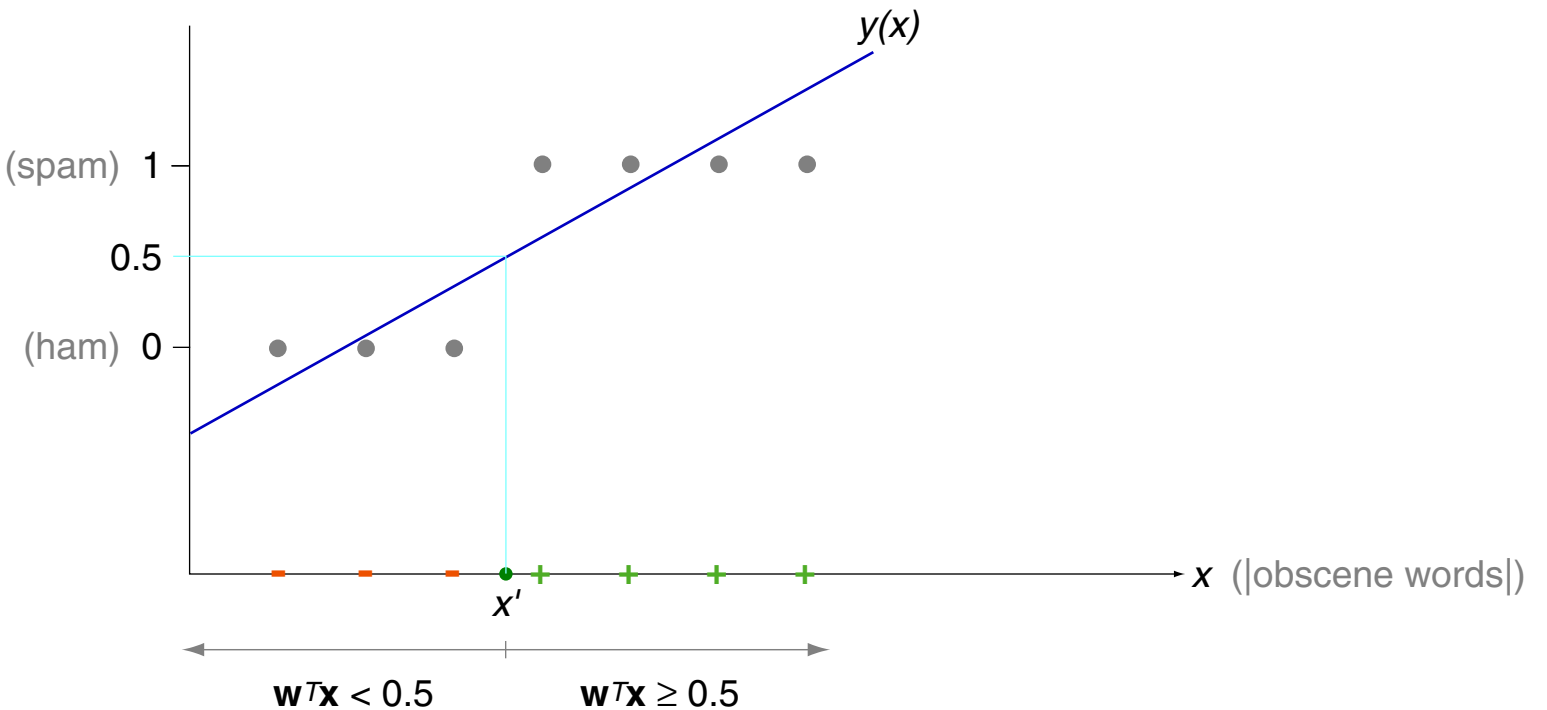


- Linear regression: $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$

©STEIN 2005-2020

Logistic Regression

Linear Regression (continued)

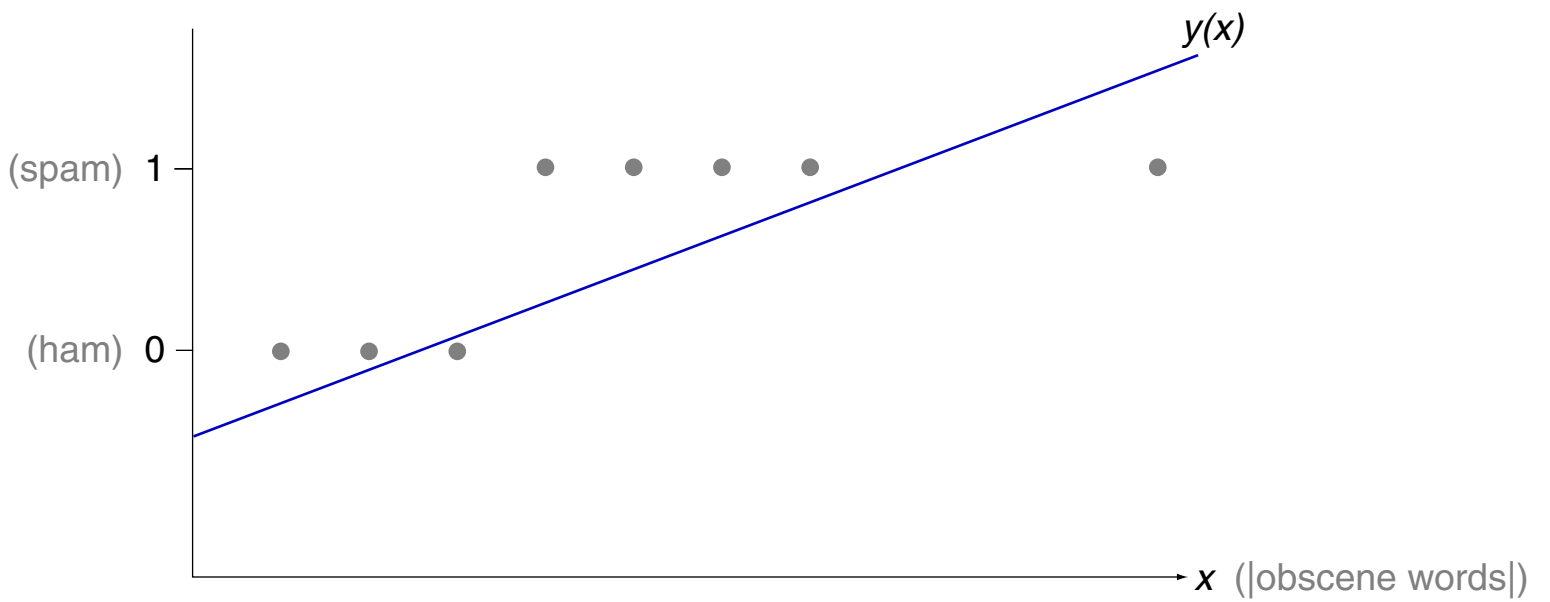


- Linear regression: $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$
- Classification: Predict $\begin{cases} \text{“spam”}, & \text{if } \mathbf{w}^T\mathbf{x} \geq 0.5 \\ \text{“ham”}, & \text{if } \mathbf{w}^T\mathbf{x} < 0.5 \end{cases}$

©STEIN 2005-2020

Logistic Regression

Linear Regression (continued)

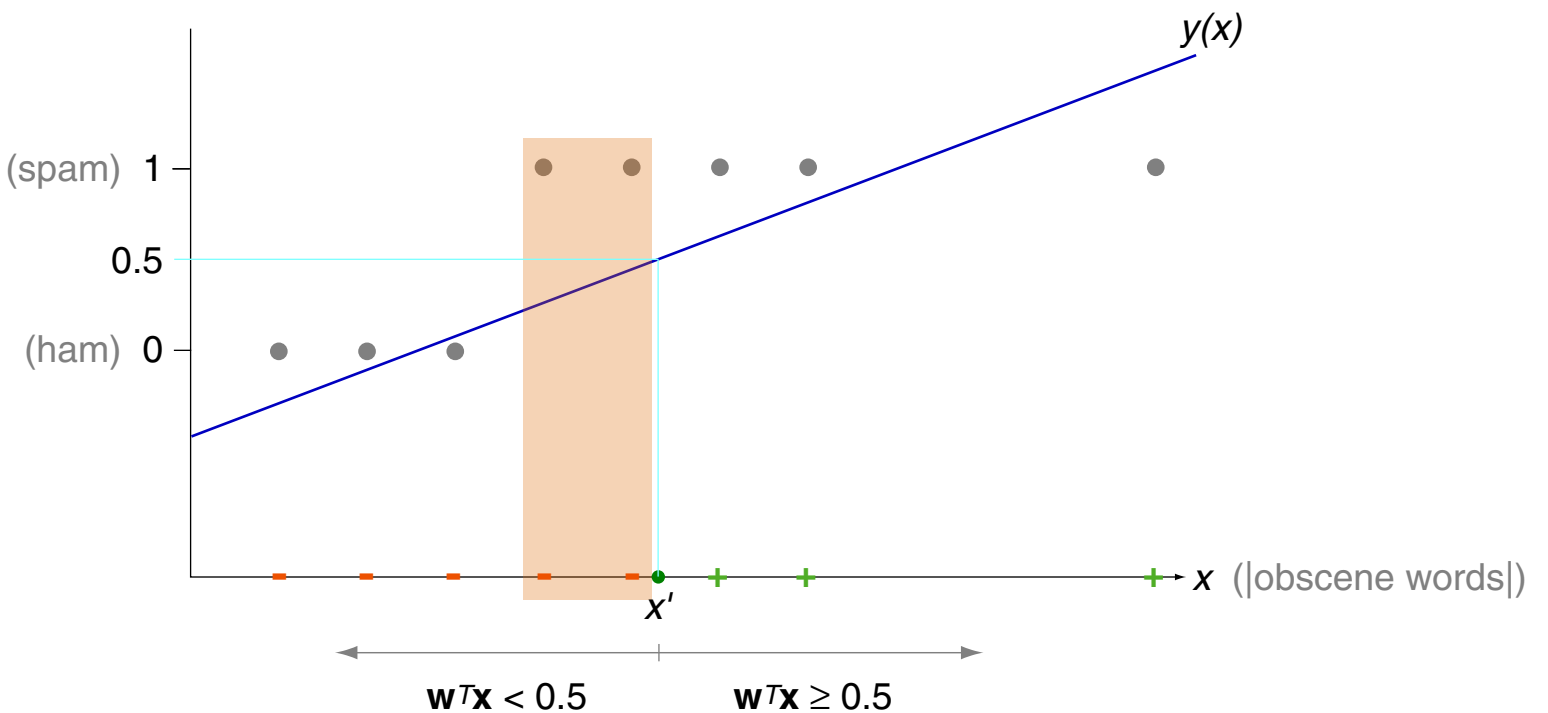


- Linear regression: $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$
- Classification: Predict $\begin{cases} \text{“spam”}, & \text{if } \mathbf{w}^T\mathbf{x} \geq 0.5 \\ \text{“ham”}, & \text{if } \mathbf{w}^T\mathbf{x} < 0.5 \end{cases}$

©STEIN 2005-2020

Logistic Regression

Linear Regression (continued)

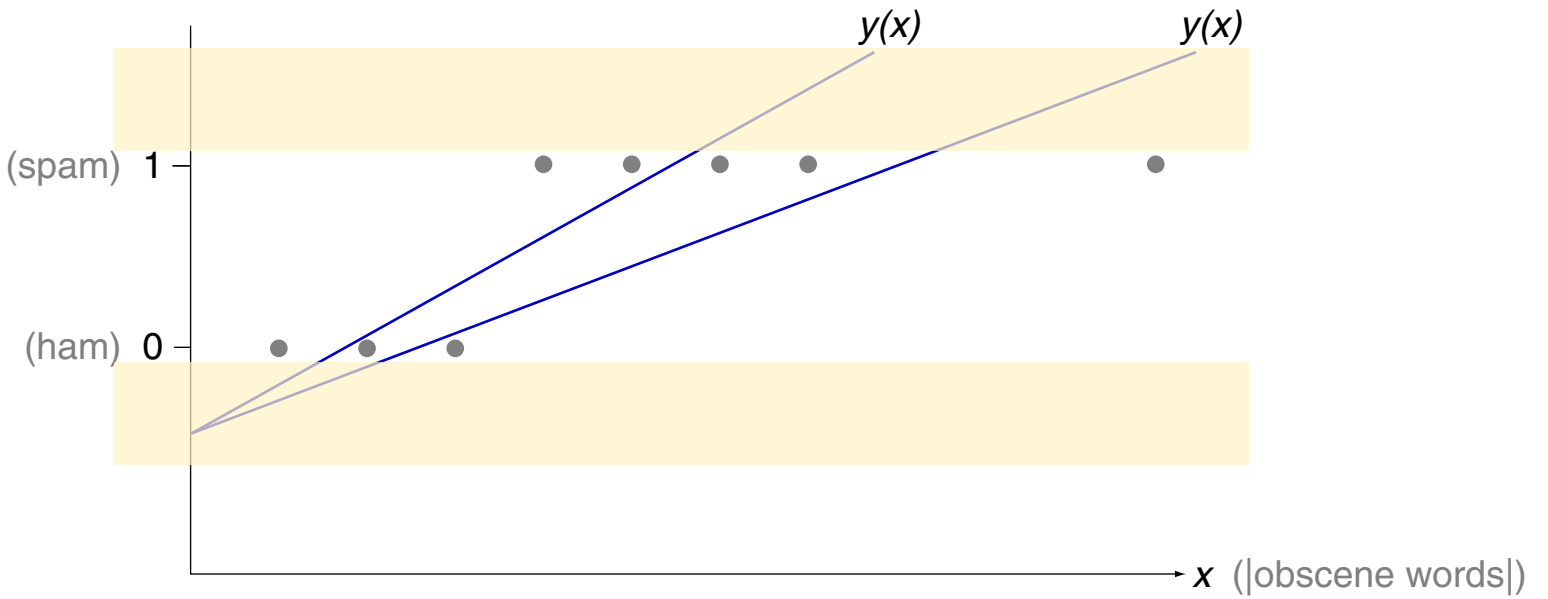


- Linear regression: $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$
- Classification: Predict $\begin{cases} \text{“spam”}, & \text{if } \mathbf{w}^T\mathbf{x} \geq 0.5 \\ \text{“ham”}, & \text{if } \mathbf{w}^T\mathbf{x} < 0.5 \end{cases}$

©STEIN 2005-2020

Logistic Regression

Linear Regression (continued)



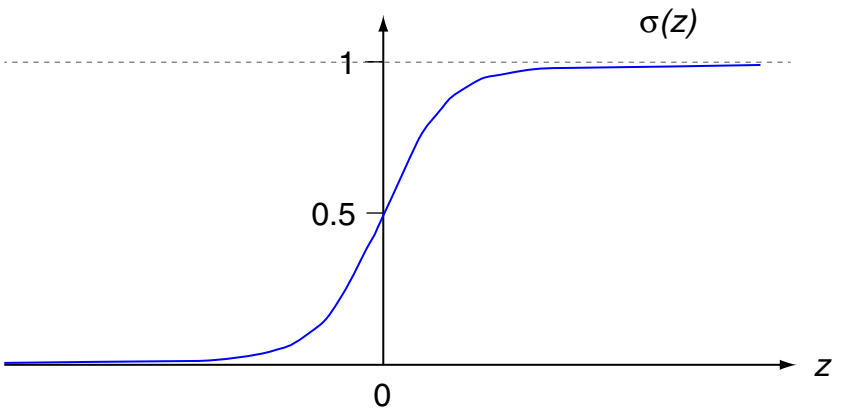
Restrict the range of $y(\mathbf{x})$ to reflect the two-class classification semantics:

$$0 \leq y(\mathbf{x}) \leq 1$$

©STEIN 2005-2020

Logistic Regression

Sigmoid (Logistic) Function



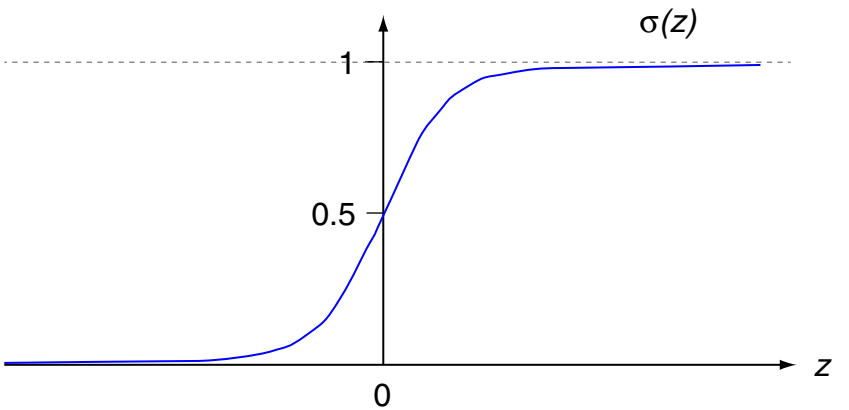
Sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

©STEIN 2005-2020

Logistic Regression

Sigmoid (Logistic) Function (continued)



Linear regression

$$\mathbf{w}^T\mathbf{x}$$

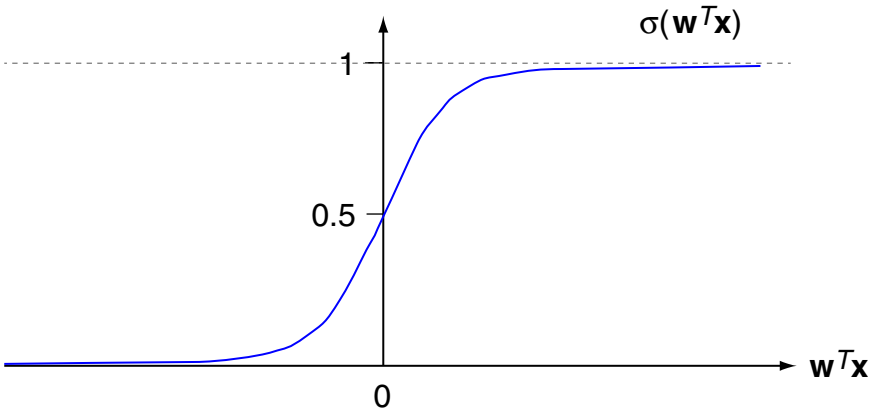
Sigmoid function

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

©STEIN 2005-2020

Logistic Regression

Sigmoid (Logistic) Function (continued)



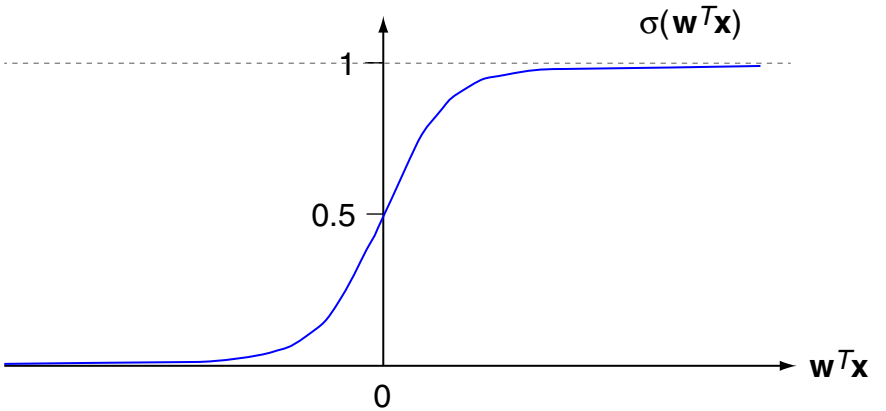
Linear regression | Sigmoid function | Logistic model function

$$\mathbf{w}^T\mathbf{x} \quad \circ \quad \sigma(z) = \frac{1}{1+e^{-z}} \quad \rightsquigarrow \quad y(\mathbf{x}) \equiv \sigma(\mathbf{w}^T\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}$$

©STEIN 2005-2020

Logistic Regression

Sigmoid (Logistic) Function (continued)



Linear regression | Sigmoid function | Logistic model function

$$\mathbf{w}^T\mathbf{x} \quad \circ \quad \sigma(z) = \frac{1}{1+e^{-z}} \quad \rightsquigarrow \quad y(\mathbf{x}) \equiv \sigma(\mathbf{w}^T\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}$$

$$y(\mathbf{x}) : \mathbf{R}^{p+1} \rightarrow (0;1)$$

©STEIN 2005-2020

Logistic Regression

Interpretation of the Logistic Model Function

$y(\mathbf{x})$ is the estimated probability that $c(\mathbf{x}) = 1$ given feature vector $\mathbf{x}$.

$y(\mathbf{x}) = P(c(\mathbf{x}) = 1 \mid \mathbf{x}; \mathbf{w})$, "Probability that $c(\mathbf{x}) = 1$ given $\mathbf{x}$, parameterized by $\mathbf{w}$."

©STEIN 2005-2020

Logistic Regression

Interpretation of the Logistic Model Function (continued)

$y(\mathbf{x})$ is the estimated probability that $c(\mathbf{x}) = 1$ given feature vector $\mathbf{x}$.

$y(\mathbf{x}) = P(c(\mathbf{x}) = 1 \mid \mathbf{x}; \mathbf{w})$, "Probability that $c(\mathbf{x}) = 1$ given $\mathbf{x}$, parameterized by $\mathbf{w}$."

Example (email spam classification):

$$\mathbf{x} = \begin{pmatrix} x_0 \\ x_1 \end{pmatrix} = \begin{pmatrix} 1 \\ |\text{obscene words}| \end{pmatrix}, \qquad \mathbf{x}_1 = \begin{pmatrix} 1 \\ 5 \end{pmatrix} \quad \text{and} \quad y(\mathbf{x}_1) = 0.67$$

$\rightsquigarrow$ 67% chance that email is spam.

©STEIN 2005-2020

Logistic Regression

Interpretation of the Logistic Model Function (continued)

$y(\mathbf{x})$ is the estimated probability that $c(\mathbf{x}) = 1$ given feature vector $\mathbf{x}$.

$y(\mathbf{x}) = P(c(\mathbf{x}) = 1 \mid \mathbf{x}; \mathbf{w})$, "Probability that $c(\mathbf{x}) = 1$ given $\mathbf{x}$, parameterized by $\mathbf{w}$."

Example (email spam classification):

$$\mathbf{x} = \begin{pmatrix} x_0 \\ x_1 \end{pmatrix} = \begin{pmatrix} 1 \\ |\text{obscene words}| \end{pmatrix}, \qquad \mathbf{x}_1 = \begin{pmatrix} 1 \\ 5 \end{pmatrix} \quad \text{and} \quad y(\mathbf{x}_1) = 0.67$$

$\rightsquigarrow$ 67% chance that email is spam.

©STEIN 2005-2020

Logistic Regression

Interpretation of the Logistic Model Function (continued)

$y(\mathbf{x})$ is the estimated probability that $c(\mathbf{x}) = 1$ given feature vector $\mathbf{x}$.

$y(\mathbf{x}) = P(c(\mathbf{x}) = 1 \mid \mathbf{x}; \mathbf{w})$, "Probability that $c(\mathbf{x}) = 1$ given $\mathbf{x}$, parameterized by $\mathbf{w}$."

Example (email spam classification):

$$\mathbf{x} = \begin{pmatrix} x_0 \\ x_1 \end{pmatrix} = \begin{pmatrix} 1 \\ |\text{obscene words}| \end{pmatrix}, \qquad \mathbf{x}_1 = \begin{pmatrix} 1 \\ 5 \end{pmatrix} \quad \text{and} \quad y(\mathbf{x}_1) = 0.67$$

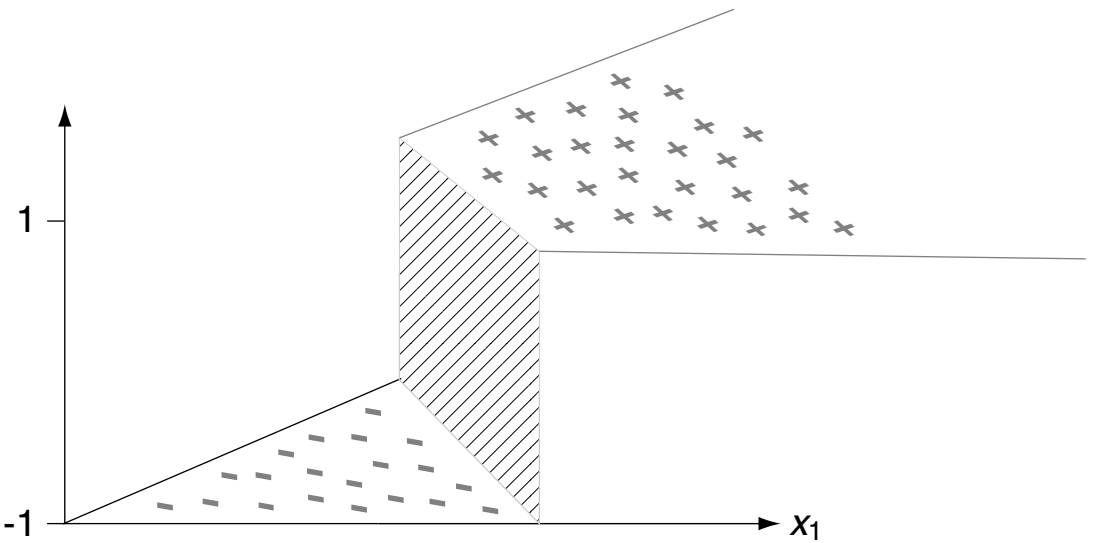
$\rightsquigarrow$ 67% chance that email is spam.

Classification: Predict $\begin{cases} c(\mathbf{x}) = 1, & \text{if } \sigma(\mathbf{w}^T\mathbf{x}) \geq 0.5 \\ c(\mathbf{x}) = 0, & \text{if } \sigma(\mathbf{w}^T\mathbf{x}) < 0.5 \end{cases}$

©STEIN 2005-2020

Logistic Regression

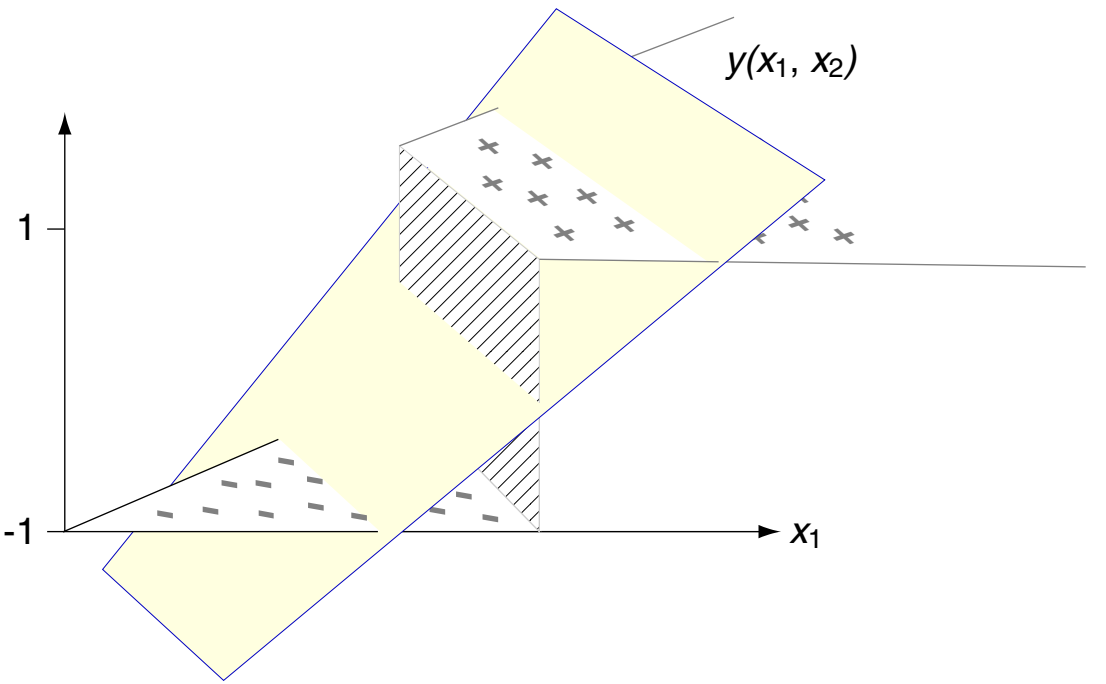
Recap: Linear Regression for Classification (illustrated for $p = 2$)



©STEIN 2005-2020

Logistic Regression

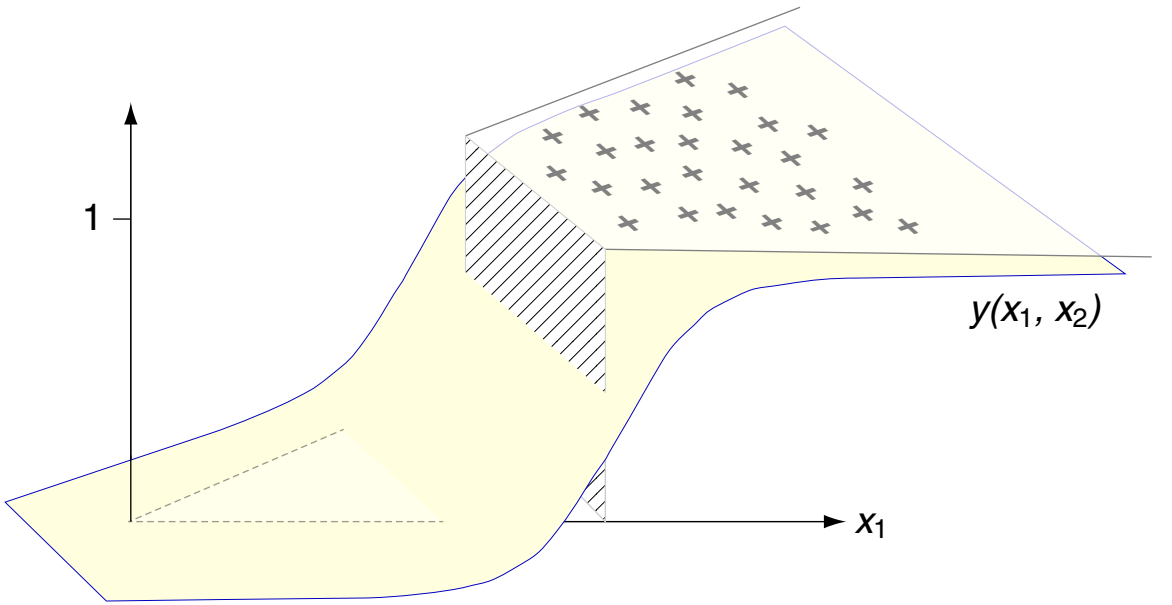
Recap: Linear Regression for Classification (illustrated for $p = 2$)



©STEIN 2005-2020

Logistic Regression

Logistic Regression for Classification (illustrated for $p = 2$)

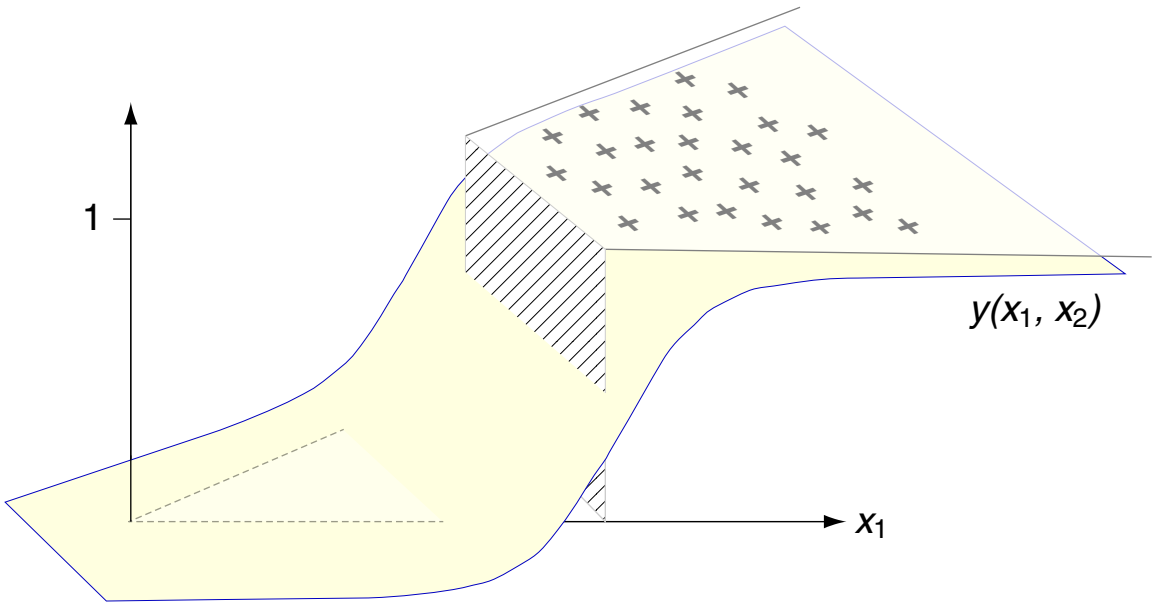


©STEIN 2005-2020

Logistic Regression

Logistic Regression for Classification (illustrated for $p = 2$) (continued)

Use logistic regression to learn $\mathbf{w}$ from D, where $y(\mathbf{x}) = \dfrac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}}$.



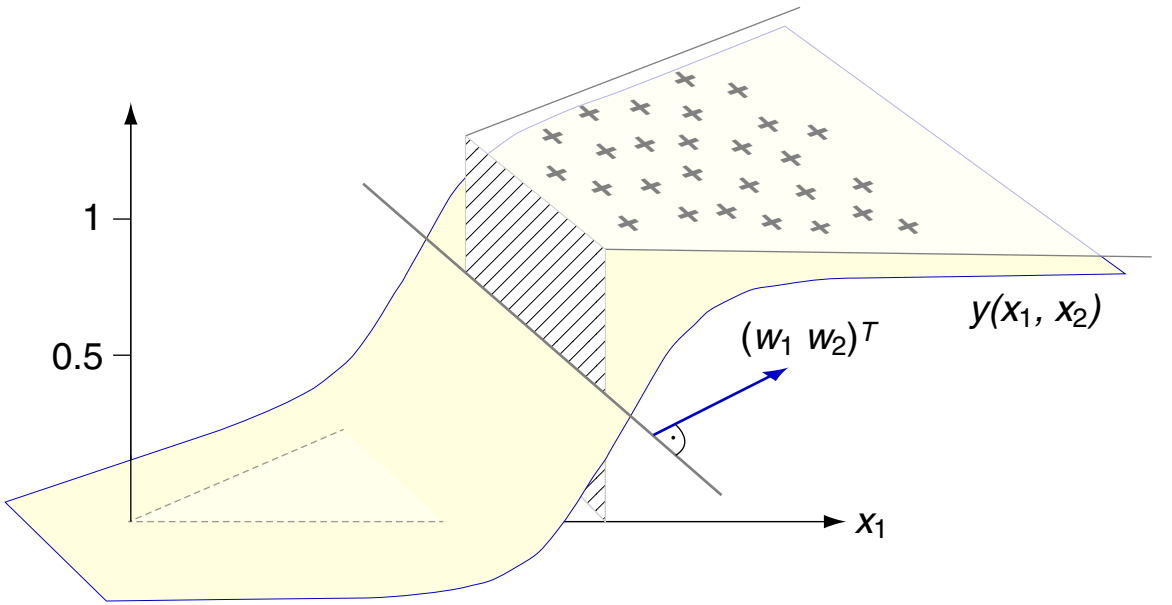
$$y(x_1, x_2) = \frac{1}{1 + e^{-(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2)}}$$

©STEIN 2005-2020

Logistic Regression

Logistic Regression for Classification (illustrated for $p = 2$) (continued)

Use logistic regression to learn $\mathbf{w}$ from D, where $y(\mathbf{x}) = \dfrac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}}$.



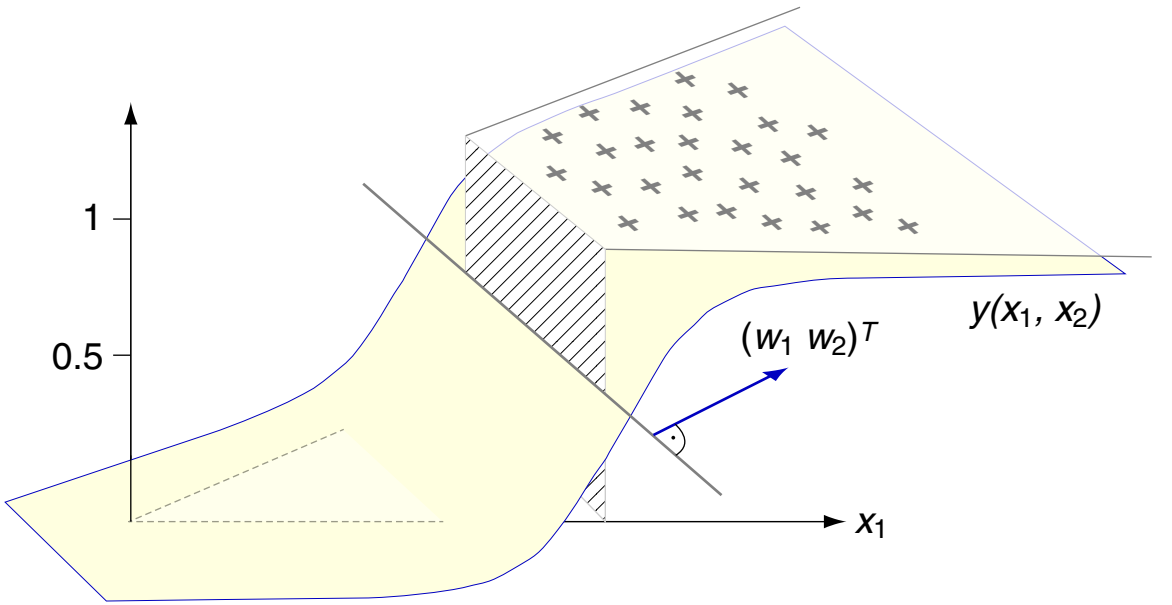
$$y(x_1, x_2) = \frac{1}{1 + e^{-(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2)}}$$

©STEIN 2005-2020

Logistic Regression

Logistic Regression for Classification (illustrated for $p = 2$) (continued)

Use logistic regression to learn $\mathbf{w}$ from D, where $y(\mathbf{x}) = \dfrac{1}{1 + e^{-\mathbf{w}^T\mathbf{x}}}$.



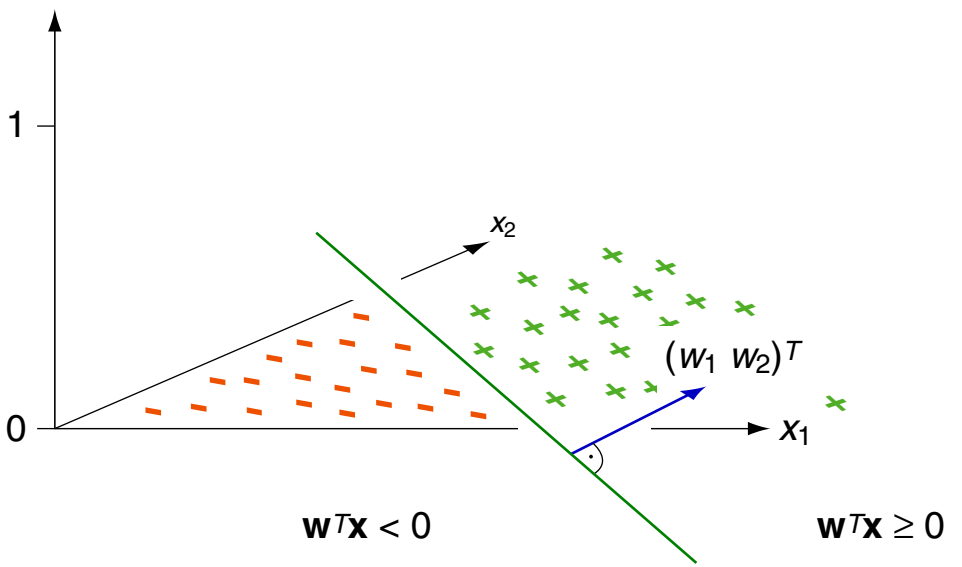
Classification: Predict $\begin{cases} c(\mathbf{x}) = 1, & \text{if } \sigma(\mathbf{w}^T\mathbf{x}) \geq 0.5 \\ c(\mathbf{x}) = 0, & \text{if } \sigma(\mathbf{w}^T\mathbf{x}) < 0.5 \end{cases}$

©STEIN 2005-2020

Logistic Regression

Logistic Regression for Classification (illustrated for $p = 2$) (continued)

Use logistic regression to learn $\mathbf{w}$ from D, where $y(\mathbf{x}) = \dfrac{1}{1 + e^{-\mathbf{w}^T\mathbf{x}}}$.



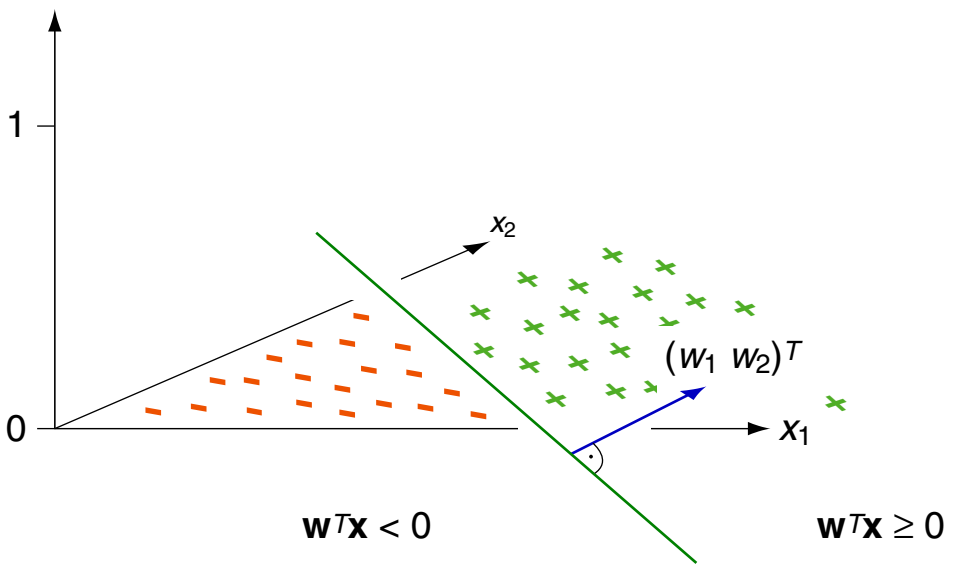
Classification: Predict $\begin{cases} c(\mathbf{x}) = 1, & \text{if } \sigma(\mathbf{w}^T\mathbf{x}) \geq 0.5 \\ c(\mathbf{x}) = 0, & \text{if } \sigma(\mathbf{w}^T\mathbf{x}) < 0.5 \end{cases}$

©STEIN 2005-2020

Logistic Regression

Logistic Regression for Classification (illustrated for $p = 2$) (continued)

Use logistic regression to learn $\mathbf{w}$ from D, where $y(\mathbf{x}) = \dfrac{1}{1 + e^{-\mathbf{w}^T\mathbf{x}}}$.

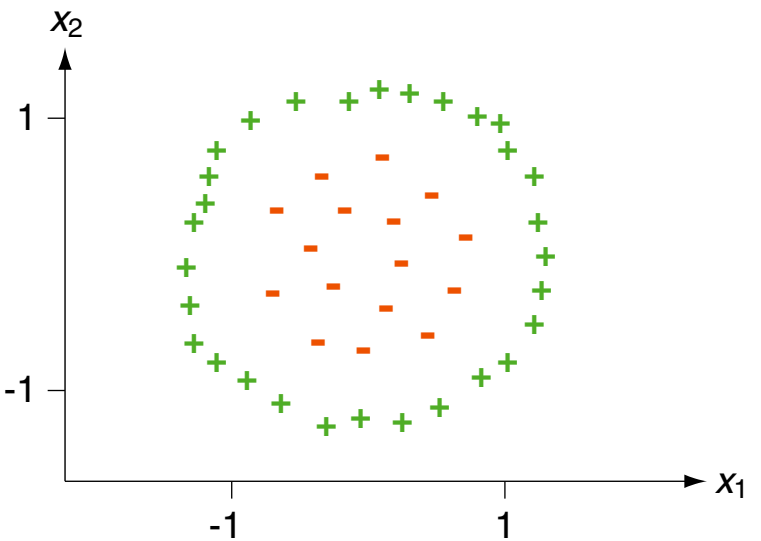


Classification: Predict $\begin{cases} c(\mathbf{x}) = 1, \text{ if } \sigma(\mathbf{w}^T\mathbf{x}) \geq 0.5 & \Leftrightarrow \mathbf{w}^T\mathbf{x} \geq 0 \\ c(\mathbf{x}) = 0, \text{ if } \sigma(\mathbf{w}^T\mathbf{x}) < 0.5 & \Leftrightarrow \mathbf{w}^T\mathbf{x} < 0 \end{cases}$

©STEIN 2005-2020

Logistic Regression

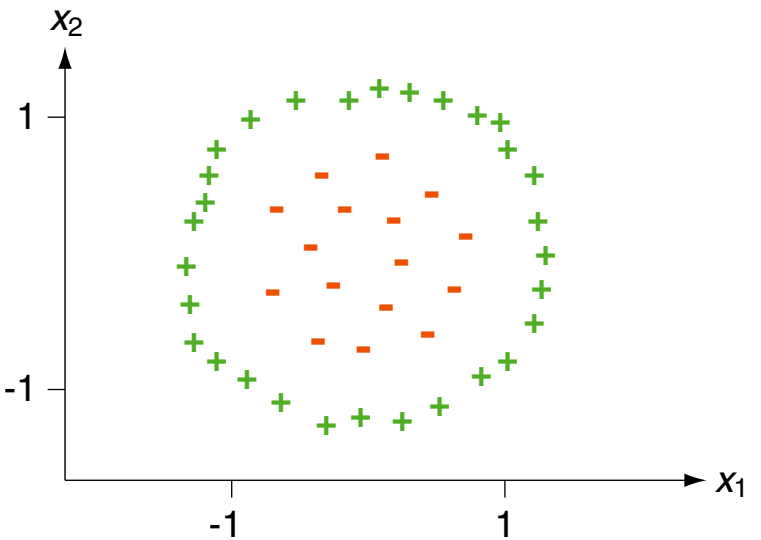
Non-Linear Decision Boundaries



©STEIN 2005-2020

Logistic Regression

Non-Linear Decision Boundaries (continued)



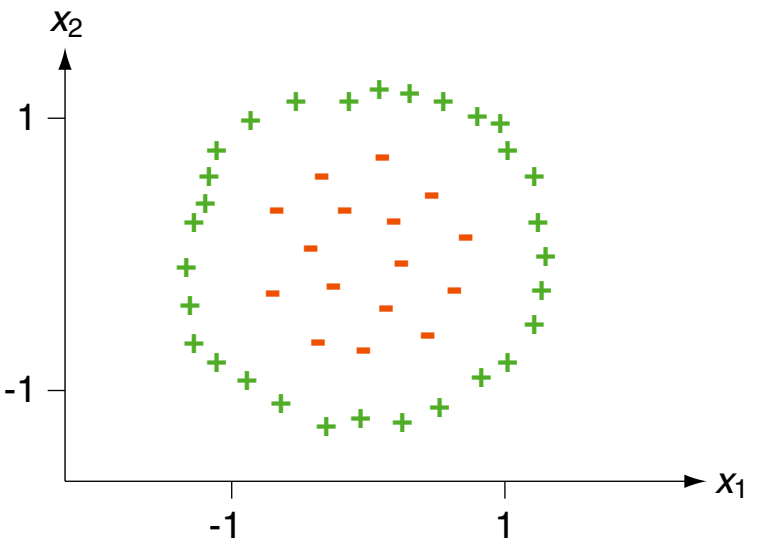
➜ Higher order polynomial terms in the features (linear in the parameters):

$$y(\mathbf{x}) = \sigma(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_1^2 + w_4 \cdot x_2^2)$$

©STEIN 2005-2020

Logistic Regression

Non-Linear Decision Boundaries (continued)



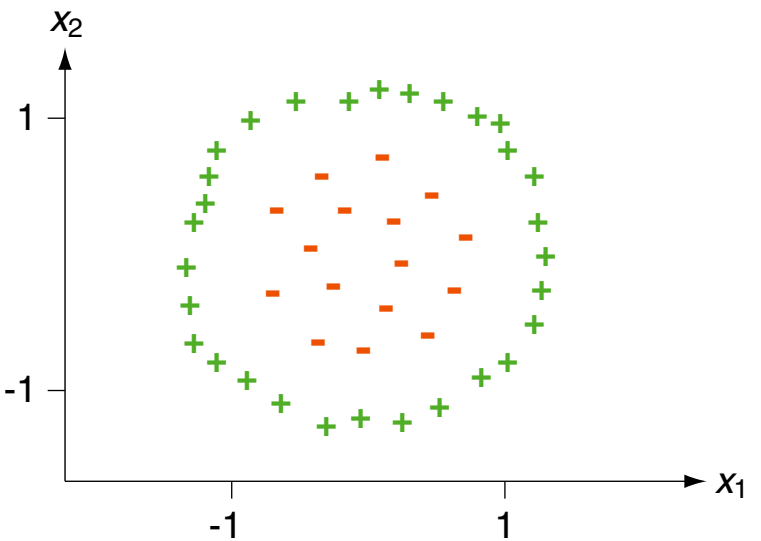
➜ Higher order polynomial terms in the features (linear in the parameters):

$$y(\mathbf{x}) = \sigma(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_1^2 + w_4 \cdot x_2^2)$$

©STEIN 2005-2020

Logistic Regression

Non-Linear Decision Boundaries (continued)



➜ Higher order polynomial terms in the features (linear in the parameters):

$$y(\mathbf{x}) = \sigma(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_1^2 + w_4 \cdot x_2^2)$$

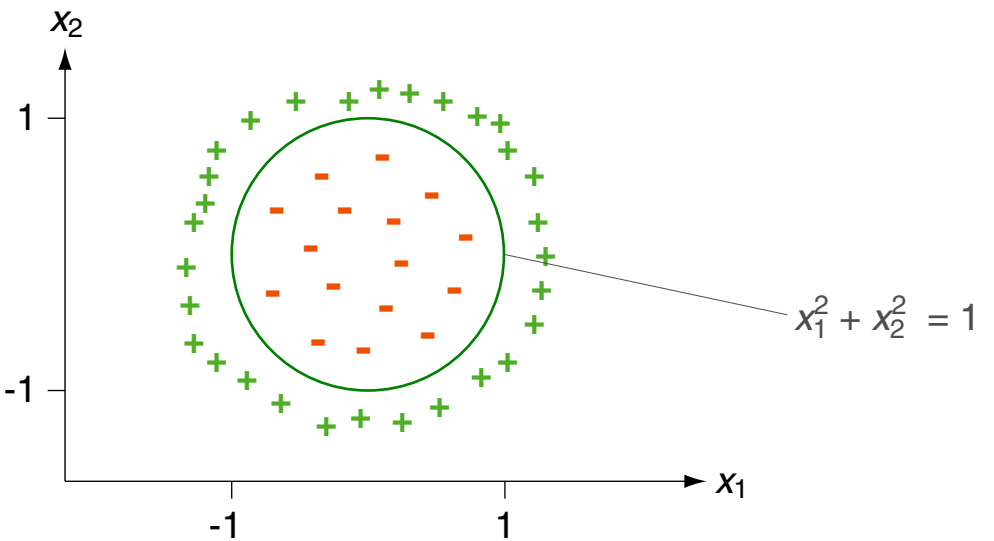
with $\mathbf{w} = \begin{pmatrix} -1 \\ 0 \\ 0 \\ 1 \\ 1 \end{pmatrix}$ $\quad \rightsquigarrow \quad y(\mathbf{x}) = \dfrac{1}{1 + e^{-(-1 \,+\, x_1^2 \,+\, x_2^2)}}$

$\rightsquigarrow$ Classification: Predict $c(\mathbf{x}) = 1$, iff $-1 + x_1^2 + x_2^2 \;\geq\; 0$

©STEIN 2005-2020

Logistic Regression

Non-Linear Decision Boundaries (continued)



➜ Higher order polynomial terms in the features (linear in the parameters):

$$y(\mathbf{x}) = \sigma(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_1^2 + w_4 \cdot x_2^2)$$

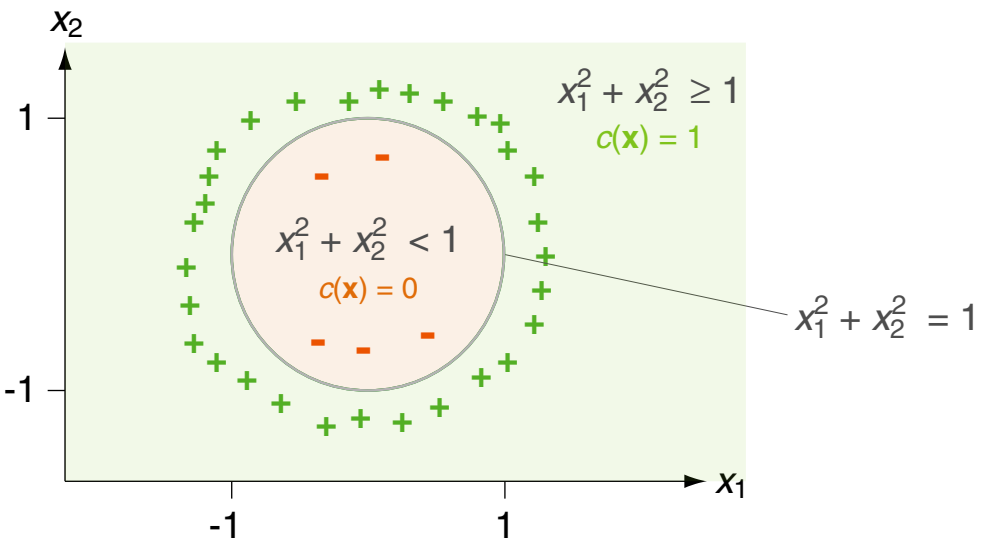
with $\mathbf{w} = \begin{pmatrix} -1 \\ 0 \\ 0 \\ 1 \\ 1 \end{pmatrix}$ $\rightsquigarrow$ $y(\mathbf{x}) = \dfrac{1}{1 + e^{-(-1 \,+\, x_1^2 \,+\, x_2^2)}}$

$\rightsquigarrow$ Classification: Predict $c(\mathbf{x}) = 1$, iff $-1 + x_1^2 + x_2^2 \;\geq\; 0$

©STEIN 2005-2020

Logistic Regression

Non-Linear Decision Boundaries (continued)



➜ Higher order polynomial terms in the features (linear in the parameters):

$$y(\mathbf{x}) = \sigma(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_1^2 + w_4 \cdot x_2^2)$$

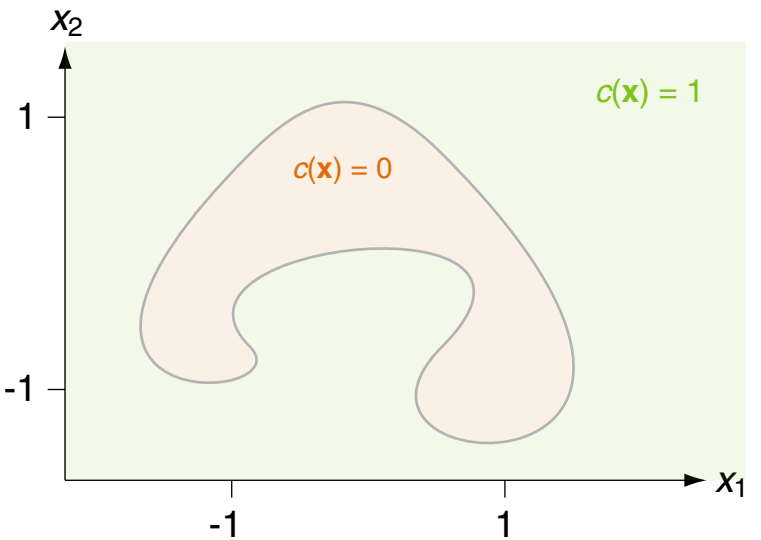
with $\mathbf{w} = \begin{pmatrix} -1 \\ 0 \\ 0 \\ 1 \\ 1 \end{pmatrix}$ $\rightsquigarrow$ $y(\mathbf{x}) = \dfrac{1}{1 + e^{-(-1 + x_1^2 + x_2^2)}}$

$\rightsquigarrow$ Classification: Predict $c(\mathbf{x}) = 1$, iff $-1 + x_1^2 + x_2^2 \geq 0$

©STEIN 2005-2020

Logistic Regression

Non-Linear Decision Boundaries (continued)



More complex polynomials entail more complex decision boundaries:

$$y(\mathbf{x}) = \sigma(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_1^2 + w_4 \cdot x_1^2 \cdot x_2 + w_5 \cdot x_1^2 \cdot x_2^2 + \ldots)$$

©STEIN 2005-2020

Remarks:

- Observe that under logistic regression the structure of a hypothesis, i.e., the forms of possible decision boundaries, is identical to the hypothesis structure under linear regression. Similarly, the respective hypothesis spaces are the same. Hence, the expressiveness, i.e., the complexity of classification problems that can be tackled (or, the effectiveness at which classification problems can be decided) is identical for the two regression approaches.

- Linear regression and logistic regression differ in the way the respective model function parameters, $\mathbf{w}$, are computed. Put another way, the strategy by which the hypothesis space is searched is different. Since both regression methods aim at loss (error, cost) minimization, their different behavior results from their different loss (error, cost) definitions.

- A note on terminology. Instead of “loss” (function) also the terms “error” (function) or “cost” (function) are used, usually with the same semantics as introduced here. We will use the term “error” for classification problems and the term “loss” for both classification and regression problems.

©STEIN 2005-2020

Logistic Regression

Loss Computation: Linear Regression

- The pointwise loss, $\ell(y(\mathbf{x}), c(\mathbf{x}))$, quantifies the penalty introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ we define the following pointwise loss functions:
  - 0/1 loss. $\ell_{0/1}(y(\mathbf{x}), c(\mathbf{x})) = I(\text{sign}(y(\mathbf{x})), c(\mathbf{x})) = \begin{cases} 0 & \text{if sign}(y(\mathbf{x})) = c(\mathbf{x}) \\ 1 & \text{otherwise} \end{cases}$
  - Squared loss. $\ell_2(y(\mathbf{x}), c(\mathbf{x})) = (y(\mathbf{x}) - c(\mathbf{x}))^2$

©STEIN 2005-2020

Logistic Regression

Loss Computation: Linear Regression (continued)

- The pointwise loss, $\ell(y(\mathbf{x}), c(\mathbf{x}))$, quantifies the penalty introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ we define the following pointwise loss functions:
  - 0/1 loss. $\ell_{0/1}(y(\mathbf{x}), c(\mathbf{x})) \;=\; I(\text{sign}(y(\mathbf{x})), c(\mathbf{x})) \;=\; \begin{cases} 0 & \text{if sign}(y(\mathbf{x})) = c(\mathbf{x}) \\ 1 & \text{otherwise} \end{cases}$
  - Squared loss. $\ell_2(y(\mathbf{x}), c(\mathbf{x})) \;=\; (y(\mathbf{x}) - c(\mathbf{x}))^2$

©STEIN 2005-2020

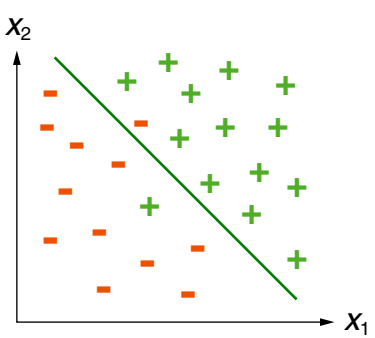
Logistic Regression

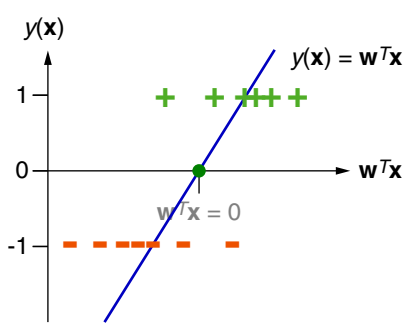
Loss Computation: Linear Regression (continued)

- The pointwise loss, $\ell(y(\mathbf{x}), c(\mathbf{x}))$, quantifies the penalty introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ we define the following pointwise loss functions:
  - 0/1 loss. $\ell_{0/1}(y(\mathbf{x}), c(\mathbf{x})) = I(\text{sign}(y(\mathbf{x})), c(\mathbf{x})) = \begin{cases} 0 & \text{if sign}(y(\mathbf{x})) = c(\mathbf{x}) \\ 1 & \text{otherwise} \end{cases}$
  - Squared loss. $\ell_2(y(\mathbf{x}), c(\mathbf{x})) = (y(\mathbf{x}) - c(\mathbf{x}))^2$

Illustration:

Input space: $y(\mathbf{x})$ over hyperplane distance:





©STEIN 2005-2020

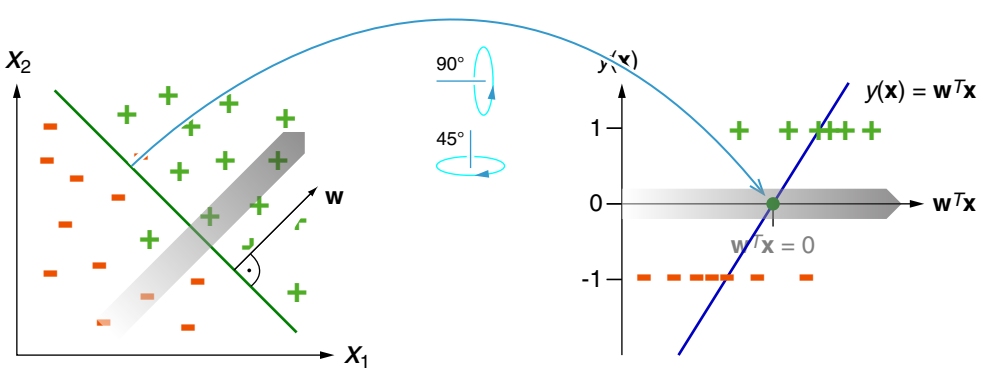
Logistic Regression

Loss Computation: Linear Regression (continued)

- The pointwise loss, $\ell(y(\mathbf{x}), c(\mathbf{x}))$, quantifies the penalty introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ we define the following pointwise loss functions:
  - 0/1 loss. $\ell_{0/1}(y(\mathbf{x}), c(\mathbf{x})) = I(\text{sign}(y(\mathbf{x})), c(\mathbf{x})) = \begin{cases} 0 & \text{if sign}(y(\mathbf{x})) = c(\mathbf{x}) \\ 1 & \text{otherwise} \end{cases}$
  - Squared loss. $\ell_2(y(\mathbf{x}), c(\mathbf{x})) = (y(\mathbf{x}) - c(\mathbf{x}))^2$

Illustration:

Input space: $y(\mathbf{x})$ over hyperplane distance:



©STEIN 2005-2020

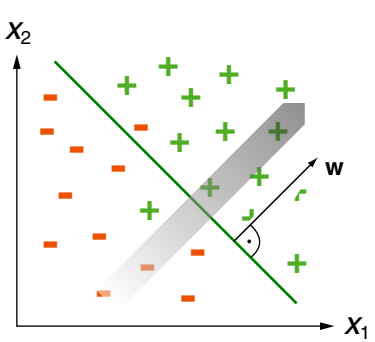
Logistic Regression

Loss Computation: Linear Regression (continued)

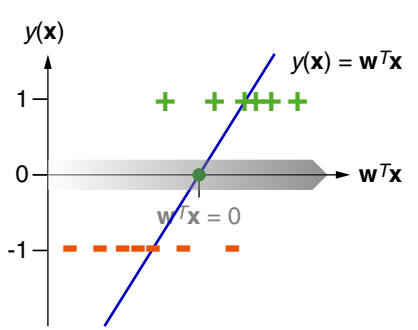
- The pointwise loss, $\ell(y(\mathbf{x}), c(\mathbf{x}))$, quantifies the penalty introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ we define the following pointwise loss functions:
  - 0/1 loss. $\ell_{0/1}(y(\mathbf{x}), c(\mathbf{x})) = I(\text{sign}(y(\mathbf{x})), c(\mathbf{x})) = \begin{cases} 0 & \text{if sign}(y(\mathbf{x})) = c(\mathbf{x}) \\ 1 & \text{otherwise} \end{cases}$
  - Squared loss. $\ell_2(y(\mathbf{x}), c(\mathbf{x})) = (y(\mathbf{x}) - c(\mathbf{x}))^2$

Illustration:

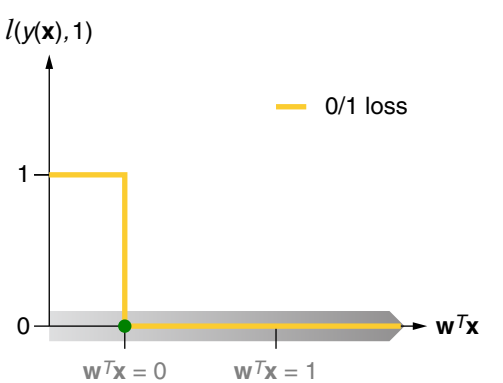
Input space:



$y(\mathbf{x})$ over hyperplane distance:



Loss over hyperplane distance:



©STEIN 2005-2020

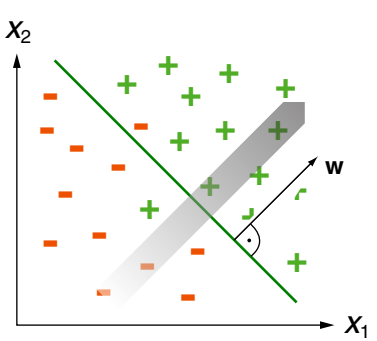
Logistic Regression

Loss Computation: Linear Regression (continued)

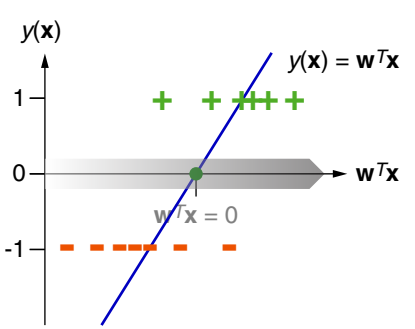
- The pointwise loss, $\ell(y(\mathbf{x}), c(\mathbf{x}))$, quantifies the penalty introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \mathbf{w}^T\mathbf{x}$ we define the following pointwise loss functions:
  - 0/1 loss. $\ell_{0/1}(y(\mathbf{x}), c(\mathbf{x})) = I(\text{sign}(y(\mathbf{x})), c(\mathbf{x})) = \begin{cases} 0 & \text{if sign}(y(\mathbf{x})) = c(\mathbf{x}) \\ 1 & \text{otherwise} \end{cases}$
  - Squared loss. $\ell_2(y(\mathbf{x}), c(\mathbf{x})) = (y(\mathbf{x}) - c(\mathbf{x}))^2$

Illustration:

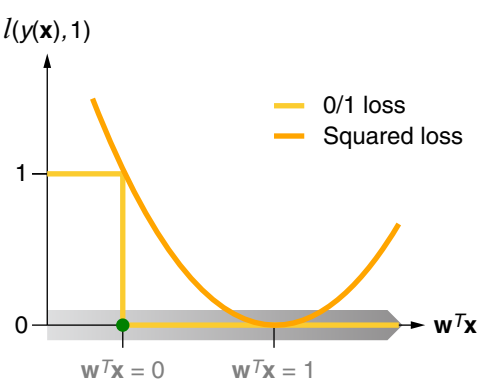
Input space:



$y(\mathbf{x})$ over hyperplane distance:



Loss over hyperplane distance:



©STEIN 2005-2020

Remarks:

- We distinguish between (1) the pointwise loss $\ell(y(\mathbf{x}), c(\mathbf{x}))$, which is computed for a single $\mathbf{x}$, and (2) the global loss $L(\mathbf{w})$, which accumulates the pointwise losses of all $\mathbf{x} \in X$ for the weight vector $\mathbf{w}$ used in a specific $y(\mathbf{x})$: $L(\mathbf{w}) = \sum_{(\mathbf{x}, c(\mathbf{x})) \in D} \ell(y(\mathbf{x}), c(\mathbf{x}))$.
- Various loss functions for classification and regression problems have been devised.
- The 0/1 loss computes the misclassification error. Recall in this regard the definition of the true misclassification rate.
- The squared loss computes the squared residual. Recall in this regard the residual sum of squares.
- As before, I denotes the indicator function.
- Regarding the illustration: $\mathbf{w}^T\mathbf{x}$ is the hyperplane distance in relation to $||\mathbf{w}||$, the length of $\mathbf{w}$. By scaling $\mathbf{w}$ such that $||\mathbf{w}|| = 1$ the hyperplane distance $\mathbf{w}^T\mathbf{x}$ becomes normalized and is also called “geometric distance”.

©STEIN 2005-2020

Logistic Regression

Loss Computation: Logistic Regression

- The pointwise loss, $\ell(y(\mathbf{x}), c(\mathbf{x}))$, quantifies the penalty introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}$ we define the following pointwise loss functions:
  - 0/1 loss. $\ell_{0/1}(y(\mathbf{x}), c(\mathbf{x})) = I(\text{sign}(y(\mathbf{x}) - 0.5), c(\mathbf{x}))$
  - Logistic loss. $\ell_{\sigma}(y(\mathbf{x}), c(\mathbf{x})) = \begin{cases} -\log(y(\mathbf{x})) & \text{if } c(\mathbf{x}) = 1 \\ -\log(1 - y(\mathbf{x})) & \text{otherwise} \end{cases}$

©STEIN 2005-2020

Logistic Regression

Loss Computation: Logistic Regression (continued)

- The pointwise loss, $\ell(y(\mathbf{x}), c(\mathbf{x}))$, quantifies the penalty introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}$ we define the following pointwise loss functions:
  - 0/1 loss. $\ell_{0/1}(y(\mathbf{x}), c(\mathbf{x})) \;=\; I(\text{sign}(y(\mathbf{x}) - 0.5), c(\mathbf{x}))$
  - Logistic loss. $\ell_{\sigma}(y(\mathbf{x}), c(\mathbf{x})) \;=\; \begin{cases} -\log(y(\mathbf{x})) & \text{if } c(\mathbf{x}) = 1 \\ -\log(1 - y(\mathbf{x})) & \text{otherwise} \end{cases}$

©STEIN 2005-2020

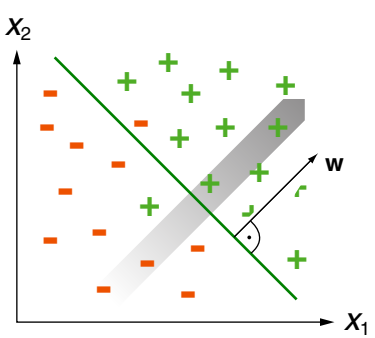
Logistic Regression

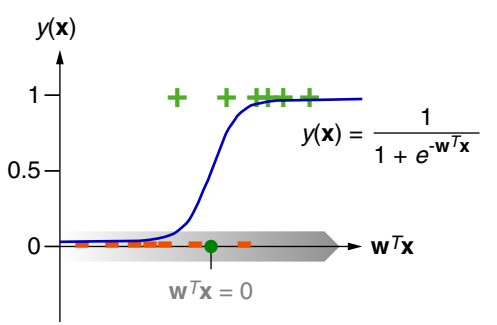
Loss Computation: Logistic Regression (continued)

- The pointwise loss, $\ell(y(\mathbf{x}), c(\mathbf{x}))$, quantifies the penalty introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}$ we define the following pointwise loss functions:
  - 0/1 loss. $\ell_{0/1}(y(\mathbf{x}), c(\mathbf{x})) = I(\text{sign}(y(\mathbf{x}) - 0.5), c(\mathbf{x}))$
  - Logistic loss. $\ell_\sigma(y(\mathbf{x}), c(\mathbf{x})) = \begin{cases} -\log(y(\mathbf{x})) & \text{if } c(\mathbf{x}) = 1 \\ -\log(1 - y(\mathbf{x})) & \text{otherwise} \end{cases}$

Illustration:

Input space: $y(\mathbf{x})$ over hyperplane distance:





©STEIN 2005-2020

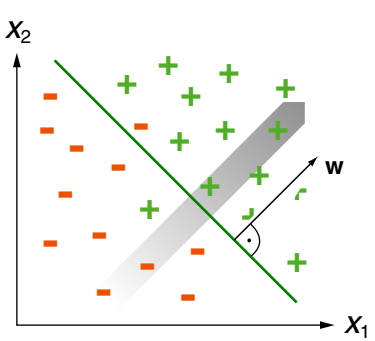
Logistic Regression

Loss Computation: Logistic Regression (continued)

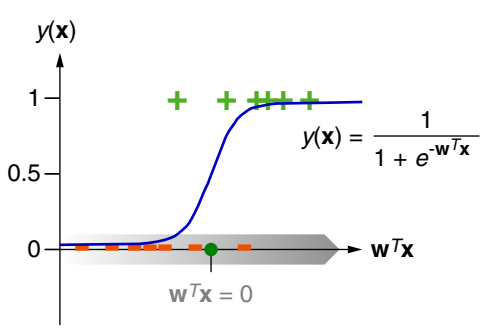
- The pointwise loss, $\ell(y(\mathbf{x}), c(\mathbf{x}))$, quantifies the penalty introduced by some $\mathbf{x}$. The loss depends on a hypothesis y and the true class, $c(\mathbf{x})$, of $\mathbf{x}$.
- For $y(\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x}) = \frac{1}{1+e^{-\mathbf{w}^T\mathbf{x}}}$ we define the following pointwise loss functions:
  - 0/1 loss. $\ell_{0/1}(y(\mathbf{x}), c(\mathbf{x})) = I(\text{sign}(y(\mathbf{x}) - 0.5), c(\mathbf{x}))$
  - Logistic loss. $\ell_{\sigma}(y(\mathbf{x}), c(\mathbf{x})) = \begin{cases} -\log(y(\mathbf{x})) & \text{if } c(\mathbf{x}) = 1 \\ -\log(1 - y(\mathbf{x})) & \text{otherwise} \end{cases}$

Illustration:

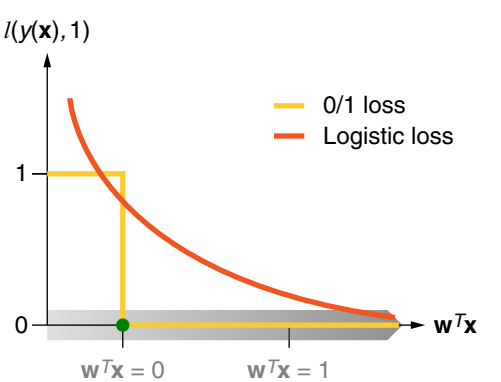
Input space:



$y(\mathbf{x})$ over hyperplane distance:



Loss over hyperplane distance:



©STEIN 2005-2020

Remarks:

- The pointwise logistic loss can be rewritten by combining the two cases algebraically:

$$\ell_\sigma(y(\mathbf{x}), c(\mathbf{x})) = -c(\mathbf{x}) \cdot \log(y(\mathbf{x})) - (1 - c(\mathbf{x})) \cdot \log(1 - y(\mathbf{x}))$$

- Observe that loss functions are employed at two places:
  1. For the fitting (the actual regression / optimization / hyperplane search) of the data, where the optimum position of the hyperplane is determined (= a particular hypothesis is selected), and,
  2. for the evaluation of a hypothesis' effectiveness, where the portion of correctly and misclassified examples is analyzed.

- Typically, fitting (optimization) and effectiveness evaluation are done with different loss functions. E.g., logistic regression uses L_σ and $L_{0/1}$ for fitting and evaluation respectively. However, linear regression (not classification) uses RSS (the L_2 loss) for both fitting and evaluation. The basic perceptron learning algorithm uses the misclassification information (the $L_{0/1}$ loss) for both fitting and evaluation.

- By now we have not specified how the parameter vector $\mathbf{w}$ is estimated under the logistic model function. In particular, there is no direct method (recall the normal equations in linear regression) to solve the model function $y(\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x})$ to obtain the optimum $\hat{\mathbf{w}}$.

©STEIN 2005-2020

Logistic Regression

Weight Adaptation: Batch Gradient Descent [Algorithms: *BGD*$_2$ *IGD* *PT*]

Algorithm:	*BGD*	Batch Gradient Descent
Input:	D	Training examples $(\mathbf{x}, c(\mathbf{x}))$ with $\lvert\mathbf{x}\rvert = p+1,\ c(\mathbf{x}) \in \{0,1\}$.
	η	Learning rate, a small positive constant.
Internal:	$y(D)$	Set of $y(\mathbf{x})$-values computed from the elements $\mathbf{x}$ in D given some $\mathbf{w}$.
Output:	$\mathbf{w}$	Weight vector with $\lvert\mathbf{w}\rvert = p+1$.

BGD(D, η)

```
initialize_random_weights(w),  t = 0
REPEAT
  t = t + 1
  Δw = 0
  FOREACH (x, c(x)) ∈ D DO
    δ = c(x) − y(x)  //  y(x) = σ(w^T x) = 1/(1+e^{−w^T x})
    Δw = Δw + η · δ · x  //  δ · x is the derivative of ℓ_σ(y(x), c(x)).
  ENDDO
  w = w + Δw
UNTIL(convergence(D, y(D)) OR t > t_max)
return(w)
```


©STEIN 2005-2020

Remarks:

- Each BGD iteration "`REPEAT ... UNTIL`" computes the global logistic loss $L_\sigma(\mathbf{w}_t)$.
- Each BGD iteration "`REPEAT ... UNTIL`" corresponds (1) to finding the direction of steepest loss descent as $-\nabla L_\sigma(\mathbf{w}_t) = \sum_{(\mathbf{x},c(\mathbf{x}))\in D} (c(\mathbf{x}) - y_t(\mathbf{x})) \cdot \mathbf{x}$, and (2) to updating $\mathbf{w}_t$ by taking a step of length η in this direction.
- The derivation of $(c(\mathbf{x}) - y(\mathbf{x})) \cdot \mathbf{x}$, which is the derivative of the pointwise loss $\ell_\sigma(y(\mathbf{x}), c(\mathbf{x}))$, is given in Section Gradient Descent, Part Neural Networks.
- The *convergence*() function may analyze the global loss, $L_\sigma(\mathbf{w}_t)$, or the norm of the loss gradient, $||\nabla L_\sigma(\mathbf{w_t})||$, and compare it to a some small positive bound ε.

©STEIN 2005-2020